Introduction and the Machine Learning Workflow

Lesson 1 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

25 September 2026 · reading time about 70 minutes

Contents

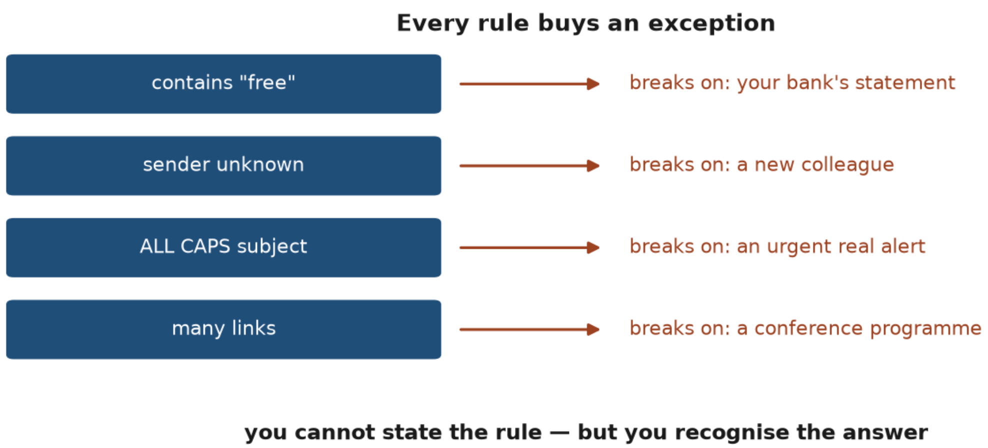
Lesson plan

Time	Segment	Material
0:00–0:15	Course introduction and assessment	Slides 1–8
0:15–0:25	Environment check	`Course/Setup/`
0:25–0:50	What learning from data means	Slides 9–22
0:50–1:05	A short history	Slides 23–30, `Resources/`
1:05–1:15	**Break**	
1:15–1:45	The three kinds of learning	Slides 31–39, notebook 02

Time	Segment	Material
1:45–2:30	The end-to-end workflow, live	Slides 40–51, notebook 01
2:30–2:55	How models mislead	Slides 52–60, notebook 03
2:55–3:00	Homework set, questions	Slides 61–62
	Total	**180 minutes**

---

1. What this course is, and what it is not



The problem you cannot specify. Every rule catches some spam and some legitimate mail, and the list never converges.

This is a course about the **foundations** of machine learning: the methods, the mathematics beneath them, and the experimental discipline needed to tell a result that holds from one that merely looks good.

It is deliberately not a tour of current AI products. Large language models, transformers and agent architectures are covered elsewhere in the programme. What you learn here is what those systems are built on, and — more importantly — how to judge whether any such system actually works.

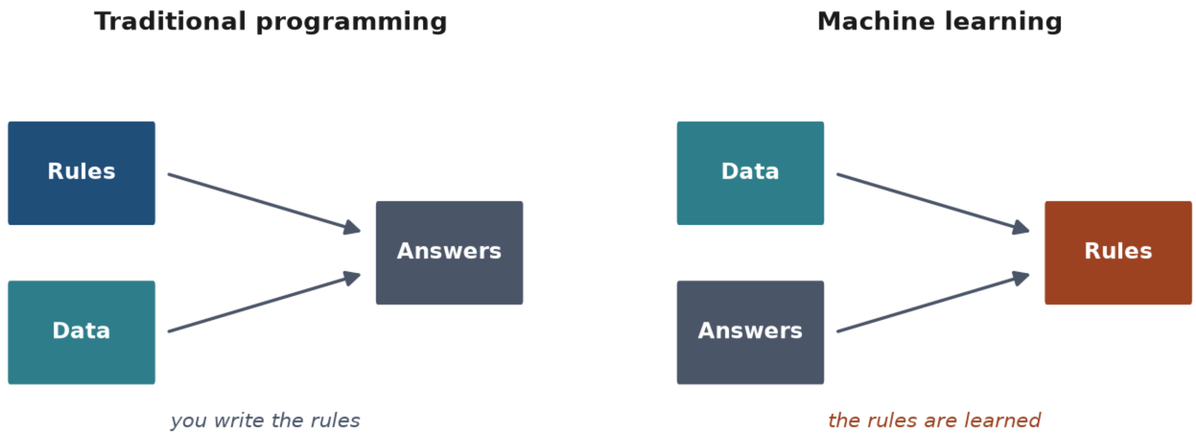
You arrive with an advantage and a gap, and it is worth naming both.

The advantage is that you can already program. You will not spend this course fighting Python, and you will be able to implement methods from scratch rather than only calling them. That matters: a method you have built once is a method you understand.

The gap is that programming ability lets you produce results faster than you can judge them. Within a few weeks you will be able to write thirty lines that yield 95% accuracy on something. The hard question — the one this course exists to answer — is whether that number means anything at all. Lesson 5 is devoted to it, and Lesson 1 begins raising it.

---

2. What learning from data means



The inversion. Programming takes rules and data and produces answers; learning takes data and answers and produces the rules.

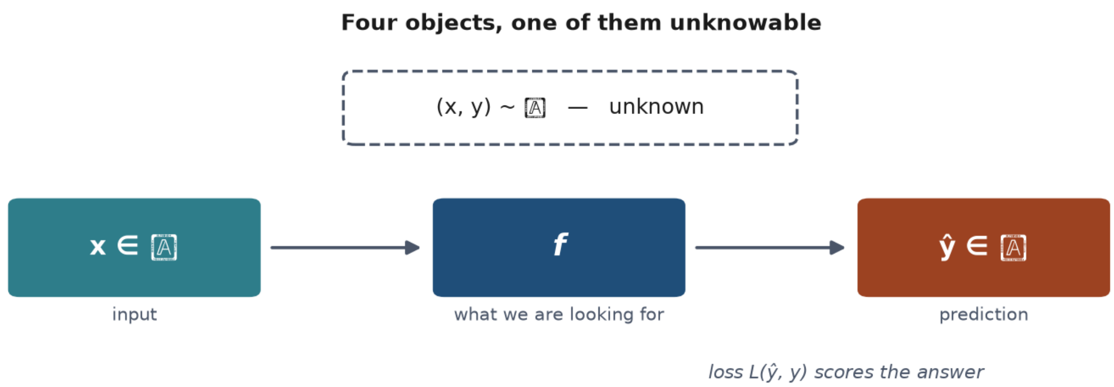
Start with the problem that machine learning solves, and notice that it is a problem about *specification*, not about computation.

Suppose you must write a program that decides whether an email is spam. You could try to write the rules by hand: flag messages containing certain words, or arriving from certain domains. You would produce something that works for a week. Spam adapts; your rules do not; the list of exceptions grows until nobody can maintain it.

The difficulty is not that the program is hard to write. It is that **you cannot state the rule**. You recognise spam instantly and cannot articulate how.

Machine learning inverts the approach. Rather than specifying the rule, you supply examples and let a procedure search for a rule consistent with them.

2.1 The formalisation



The setup in one picture: an unknown distribution, a sample drawn from it, a model chosen using that sample, and the world it will actually meet.

This much can be made precise, and doing so pays off later.

We have an input space $\mathcal{X}$ (emails, tumour measurements, images) and an output space $\mathcal{Y}$ (spam or not, malignant or benign, a price). We assume pairs (x, y) are drawn from some fixed but unknown probability distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$.

We want a function $f : \mathcal{X} \to \mathcal{Y}$ that predicts well. "Well" needs a definition, so we introduce a **loss function** $L(\hat{y}, y)$ measuring the cost of answering $\hat{y}$ when the truth is y. The quantity we actually care about is the **expected risk**:

$$R(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\left[L(f(x), y)\right]$$

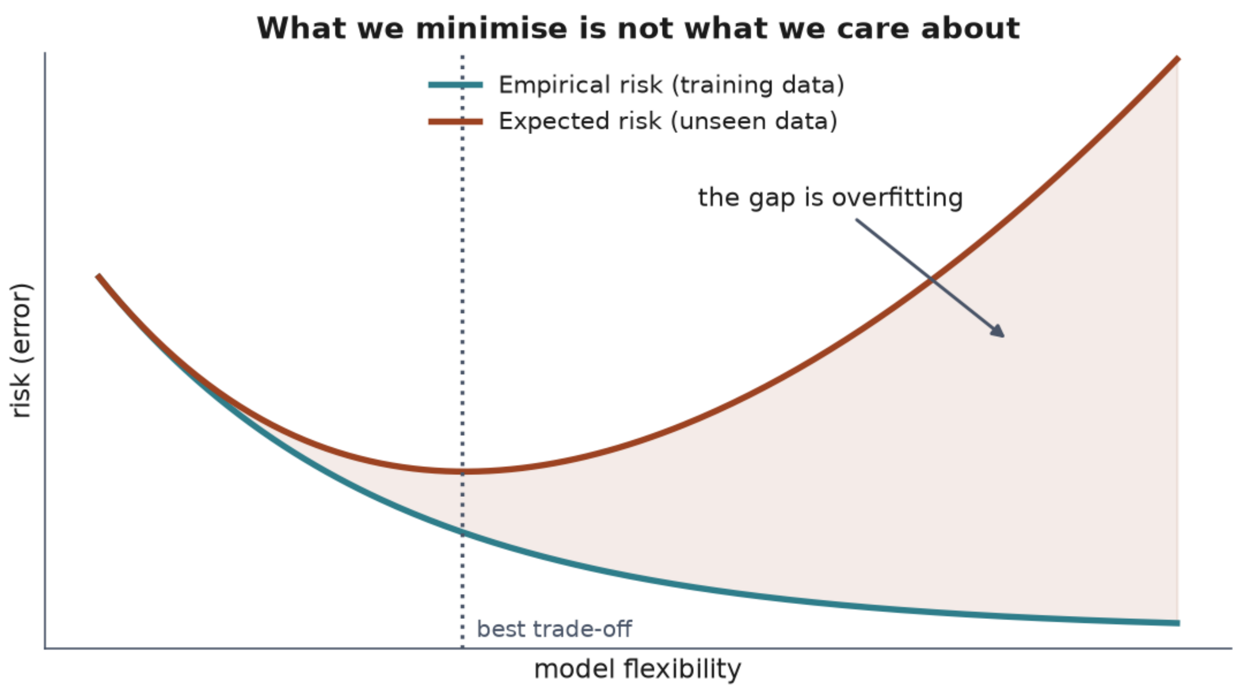
This is the average loss over all data the world might produce — including data that does not exist yet. It is the thing we want to make small.

And it is **not computable**, because $\mathcal{D}$ is unknown. All we hold is a finite sample $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$, so we compute the **empirical risk** instead:

$$\hat{R}_S(f) = \frac{1}{m}\sum_{i=1}^{m} L(f(x_i), y_i)$$

and choose the f that makes it small. This is *empirical risk minimisation*, and essentially every method in this course is an instance of it.

2.2 The gap that explains the whole course



The quantity we want and the quantity we can compute are not the same. Everything in this course is about keeping the gap between them small and honest.

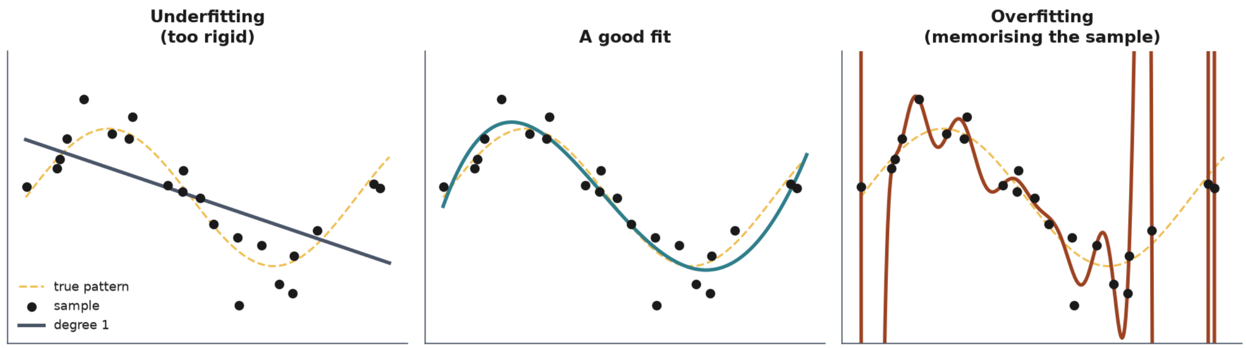
Here is the crux. We minimise $\hat{R}_S(f)$ but care about $R(f)$, and the two are not the same number.

A sufficiently flexible f can drive the empirical risk to zero by memorising the sample — storing every pair and reciting the answer. Its expected risk would be terrible: it has learnt the sample rather than the pattern. That is **overfitting**, and it is the central difficulty of the field.

So the practical question is never "how well does the model do on the data I fitted it to?" — that number is always optimistic and sometimes meaningless. It is "how well will it do on data it has never seen?"

Since $R(f)$ cannot be computed, we estimate it: hold out part of the sample, never let the fitting procedure touch it, and measure there. **That is the entire reason the test set exists.**

2.3 Why holding data out actually works



The same idea on real data: the error on data the model has seen keeps falling, while the error on data it has not turns around.

The picture first. Imagine a revision session the week before an exam, in which the lecturer works through exactly the exercises that will be on the paper — not deliberately, simply because those are the exercises on their mind. The class does very well. Nobody cheated, and every mark was earned. But the marks now measure how well those particular exercises were rehearsed rather than how well the subject is understood, and nothing in the marks themselves tells the two apart.

A test set is that exam paper. It measures generalisation only for as long as nothing has rehearsed on it — for as long as nothing about it influenced the model. That is the entire content of the rule, and the argument below is that sentence made precise.

The rule is easy to state and easy to treat as hygiene. It is worth seeing why it is not — the justification is short, and it tells you exactly when the rule has been broken.

The difficulty is not that a sample is small or unrepresentative. It is this:

> If the same data both **chooses** the model and **judges** it, the judgement is no longer an unbiased estimate. It is systematically optimistic.

An analogy that lands in a lecture: the exercises rehearsed in the revision session cannot also be the exercises that are marked. Nobody cheated — the same questions simply did two jobs at once, and the mark inherited the optimism.

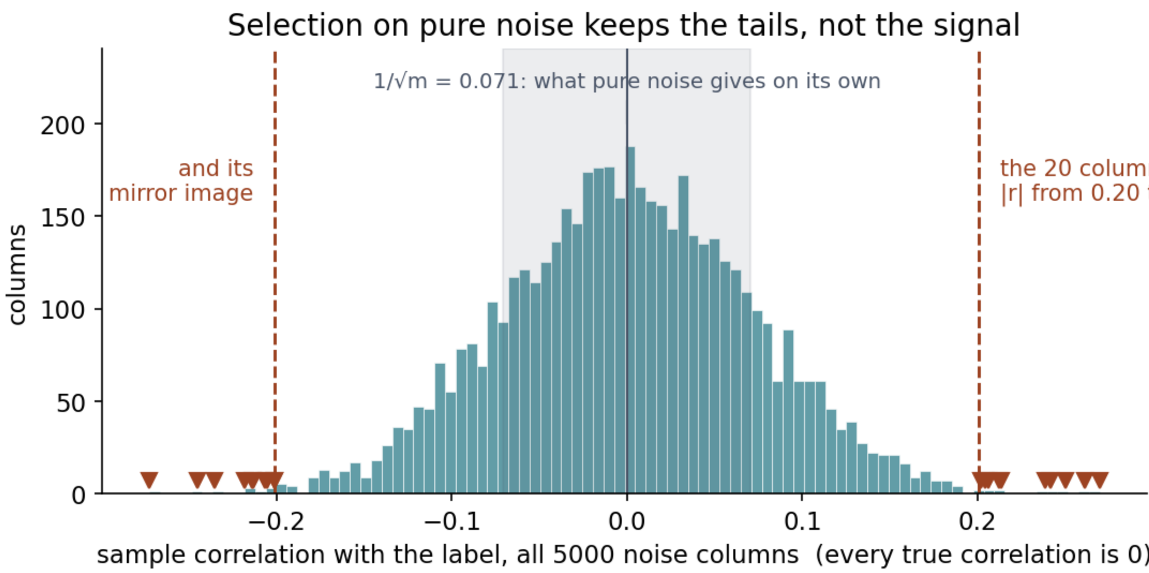
Now the reason a held-out set repairs this. Suppose we fix a function f using the training data alone, and then evaluate it on a test set T drawn from the same distribution $\mathcal{D}$ and never consulted while choosing f. Because T is independent of f, each test example is an unbiased draw of the loss, and so

$$\mathbb{E}_{T\sim\mathcal{D}^m}\left[\hat{R}_T(f)\right] = R(f)$$

The empirical risk on the test set is an **unbiased estimator of the expected risk** — the quantity we said was not computable. That equation is what the test set buys, and it is worth writing on the board.

Notice precisely what the argument depends on: **the independence of T from f.** Not on the size of the split, not on the proportion, not on stratification. The moment the test rows influence any decision — a mean used for scaling, a ranking used to select features, a comparison used to pick a model — f ceases to be independent of T, the equality above fails, and the estimate becomes optimistic by an unknown amount.

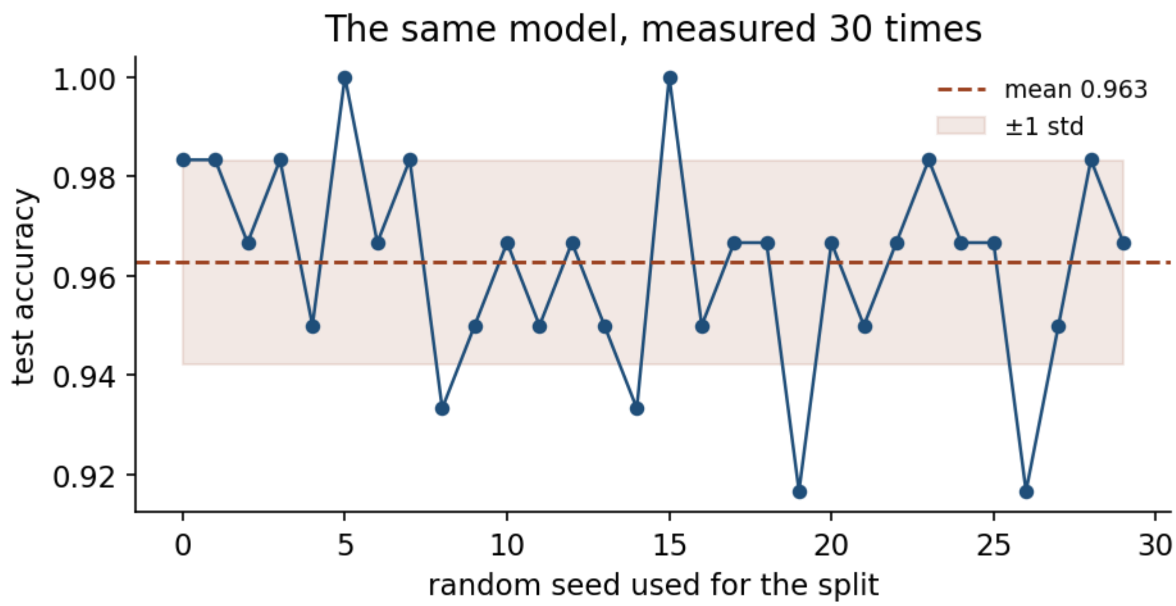
That is why the rule is "nothing is learned from the data before the split", not "remember to keep some data aside". Notebook 03 breaks exactly this independence and obtains **77% accuracy on labels generated by a coin flip**.



How that happens, on the very data that produces the 77%. Every one of the 5000 columns has a true correlation of zero with the label; a sample correlation measured on 200 rows lands within about $1/\sqrt{m} = 0.07$ *of zero, which is the shaded band. Look at where the triangles are: the twenty columns the selector kept are the extremes of that distribution, in both tails, with no signal in any of them. And those extremes were read off all 200 rows — including the 60 that were about to become the test set.*

The test information did not enter through the rows. It entered through the choice of columns, which is why nothing in the code looks wrong.

2.4 How much to hold out, and how much to trust it



One split is one measurement. Repeat it with a different seed and the number moves — which is why lesson 5 is about measuring properly.

The picture first. A test score is a measurement, and measurements have error bars. Ask a hundred people whether they will vote for a party and you would not report the result to three decimal places; ask a thousand and the figure steadies. A test set works the same way: the fewer examples it holds, the noisier the number it gives you.

That is where the trade-off comes from. Move examples into the test set and the measurement steadies but the model has less to learn from; move them the other way and you get a better model whose quality you know less precisely.

Two practical consequences follow, and both surprise people.

The estimate has a variance of its own. An accuracy measured on m test examples is a proportion, so its standard error is roughly

$$\mathrm{SE} \approx \sqrt{\frac{p(1-p)}{m}}$$

In Notebook 01 the test set holds 143 examples and the accuracy comes out at 0.986. That gives a standard error of about **one percentage point**, so reporting "0.986" to three decimal places claims a precision we do not have: the third digit is noise. This is the same observation that Section 7 makes about single splits, arriving from the other direction.

And the formula has a limit worth meeting here. It is a normal approximation, and a normal approximation needs enough events on both sides to be trusted — the usual rule of thumb asks for at least five. But 0.986 of 143 is **two errors**, and $m\,p(1-p) \approx 2$. Push the approximation anyway and it returns an interval running up to **100.5%**: it admits accuracies above 100%, which settles the matter without any further argument. A method that does not lean on the approximation puts the lower bound at **95.0%** instead — a point and a half below what the approximation promised, and in the flattering direction. Lesson 5 takes up how such an interval is properly built. What belongs here is the habit: ask whether a formula is admissible before quoting the number it gives you.

Hence the trade-off in choosing the split. A larger test set gives a more stable estimate — the standard error falls as $1/\sqrt{m}$. A larger training set gives a better model. With thousands of examples, holding out 20-30% costs little and buys a reliable number. With a few hundred, both sides hurt at once, which is precisely the situation cross-validation is designed for. Lesson 5 takes it up.

2.5 One simplification we are making today

Train and test today; lesson 5 adds the validation set that every choice should actually be made on.

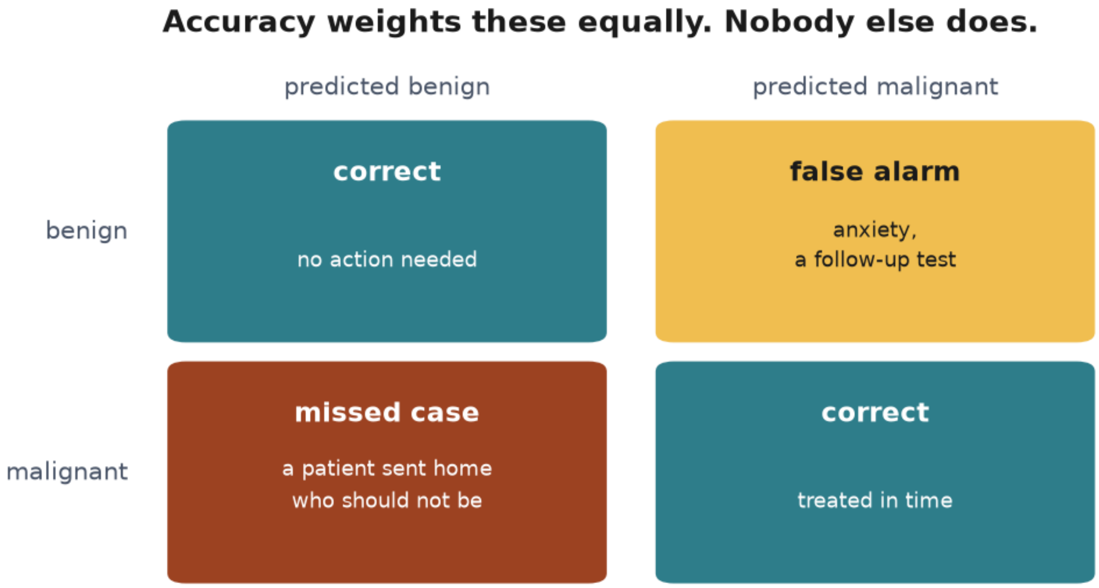
This lesson speaks of a training set and a test set. In practice three roles are needed:

Set	Used for	How often you may look
Training	Fitting the model	Freely
Validation	Choosing between models and settings	Freely
Test	The final reported number	Once

Today we do not need the middle one, because we make no choices: one model, no tuning, no comparison. As soon as you compare ten candidates and report the best, you have *selected* using whatever data you compared them on — and if that was the test set, the winning number is optimistic again, for exactly the reason in Section 2.3. The selection is a decision, and decisions are what the test set must stay independent of.

Everything in Lesson 5 elaborates these three sections.

3. When not to use machine learning



Framing the problem means pricing the two errors before choosing a model, because that price is what decides what "good" means.

An unfashionable section, and the most immediately useful one.

When the rule is known. If you can state the rule, write it. Nobody should train a classifier to detect whether a number is even. This sounds obvious and is violated constantly, usually because ML is the interesting option.

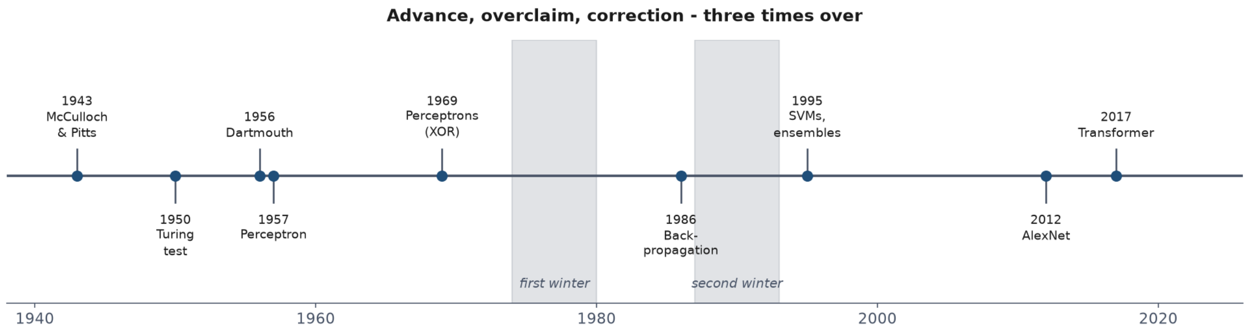
When you cannot obtain representative data. A model learns the distribution it was trained on. If your sample comes from a different population than the one you will deploy on — different hospital, different season, different user base — the model's measured performance says little about its real performance.

When errors are catastrophic and unexplainable. Learned models are wrong sometimes, in ways that are hard to predict and often hard to explain. If a mistake is unrecoverable, and no human reviews the output, the question is not whether the accuracy is high enough but whether anyone can tell when it has failed.

When the data encodes an injustice you would be automating. If historical decisions were biased, a model fitted to them reproduces the bias — with an appearance of objectivity that makes it harder to contest. The model is not neutral because it is mathematical; it is a compressed summary of the decisions in its training data.

When a simple baseline is already sufficient. Often a threshold on one variable solves 90% of the problem, is understandable by everyone, and needs no maintenance. Establishing baselines first (Section 6) is partly a way of discovering this before building something complicated.

4. A short history, and what it teaches



Seventy years in one picture. The pattern worth recognising is the rhythm: a genuine advance, claims well beyond the evidence, and a correction expensive enough to cost a generation of funding.

The field's history is worth an hour not for the anecdotes, but because its pattern repeats and you are living through another iteration of it.

1943 — the first artificial neuron. McCulloch and Pitts model a neuron as a threshold unit: it fires when the weighted sum of its inputs exceeds a threshold. The model is a claim that thought might be computation.

1950 — Turing's imitation game. Turing sidesteps "can machines think?" and replaces it with a question that can be tested by observation — an early instance of the move this course keeps making: replace an unanswerable question with a measurable proxy, and stay aware of the gap between them.

1956 — Dartmouth. The term *artificial intelligence* is coined at a summer workshop whose proposal suggests that significant progress could be made in two months by ten people. The optimism is not incidental; it is the pattern.

1957 — the perceptron. Rosenblatt builds a learning machine: weights adjusted from examples rather than set by hand. It is the direct ancestor of everything in Lessons 9 and 10. Press coverage promises machines that will walk, talk and be conscious.

1969 — the first winter begins. Minsky and Papert prove the perceptron cannot represent XOR. The limitation is real but narrower than the reception suggests — it applies to a single layer, and multi-layer networks were not yet trainable. Funding collapses for a decade.

1986 — backpropagation popularised. Rumelhart, Hinton and Williams give a practical way to train multi-layer networks, removing the objection. Enthusiasm returns; a second winter follows in the early 1990s when results again fall short of promises.

1990s–2000s — statistics takes over. Support vector machines, ensembles and probabilistic methods dominate. The framing shifts from "simulating intelligence" to "estimating functions from data" — a retreat in ambition and an advance in rigour. Most of this course lives here.

2012 — deep learning arrives. AlexNet wins ImageNet by a wide margin. The ideas are largely from the 1980s; what changed is data and compute. This is worth pausing on: the breakthrough came from scale, not from a new principle.

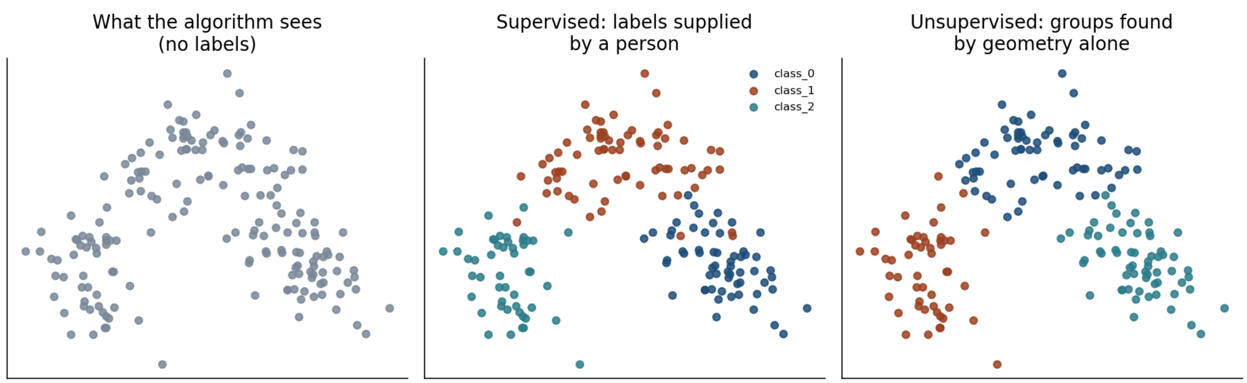
2017 onwards — scale. The transformer architecture, then models trained on internet-scale corpora. Capabilities that surprised most researchers, alongside claims that would have been familiar to anyone reading press coverage of the perceptron.

What the pattern teaches. Each cycle featured genuine advances, followed by claims outrunning evidence, followed by correction. The technical lesson is that progress has come as much from data and computation as from new ideas. The professional lesson is that the ability to distinguish a demonstrated result from an extrapolated promise is a durable skill — and it is, again, the skill this course is built to teach.

`Resources/history_of_ai.md` covers this in more depth, with the primary sources.

---

5. The three kinds of learning



One dataset, three questions. What changes is not the data but what you ask of it.

The standard taxonomy divides methods by **where the target comes from**. That is the distinction that matters; the algorithms often overlap.

5.1 Supervised learning

Each example carries a label somebody produced: (x_i, y_i) pairs. The model learns the mapping and can be checked against ground truth.

When $\mathcal{Y}$ is a finite set of categories the task is **classification**; when it is continuous, **regression**. Lessons 3, 4, 6 and 7 are all supervised.

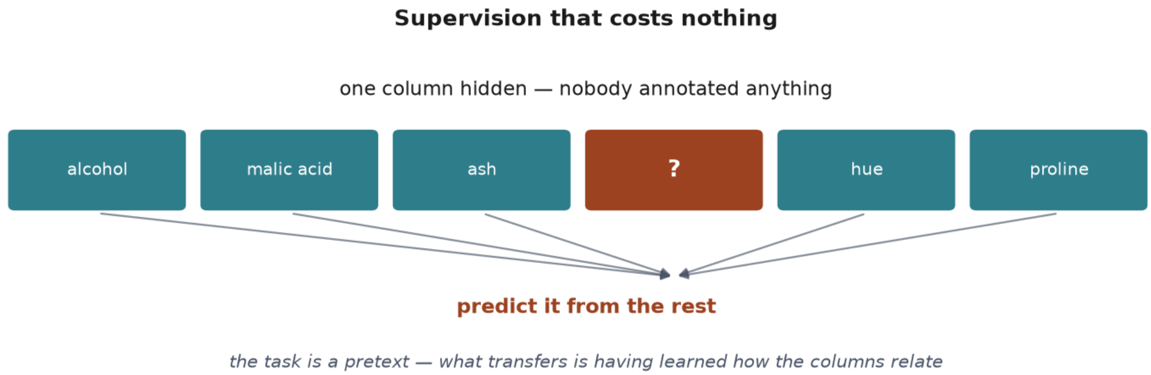
The cost is the labels. In practice this — not algorithms, not compute — is what limits most projects.

5.2 Unsupervised learning

No targets. The data is $\{x_1, \ldots, x_n\}$ and the goal is structure: groups (clustering), a lower-dimensional description (dimensionality reduction), or unusual points (anomaly detection). Lesson 8.

There is no accuracy to report, which is the difficulty. An algorithm will always return groups. Whether they correspond to anything you care about is a judgement you must make, and it cannot be delegated to a metric.

5.3 Self-supervised learning



Where the labels come from when nobody labelled anything: hide part of the input and ask the model to reconstruct it.

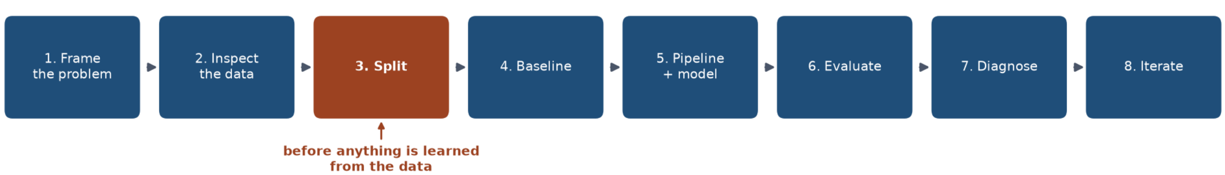
A target is manufactured from the input: hide part of each example and train the model to reconstruct it from the rest. Nobody annotates anything, yet the supervision is genuine.

The reconstruction task itself is of no interest — it is a *pretext*. The point is that solving it forces the model to represent how the parts of an input relate, and that representation transfers to tasks you do care about.

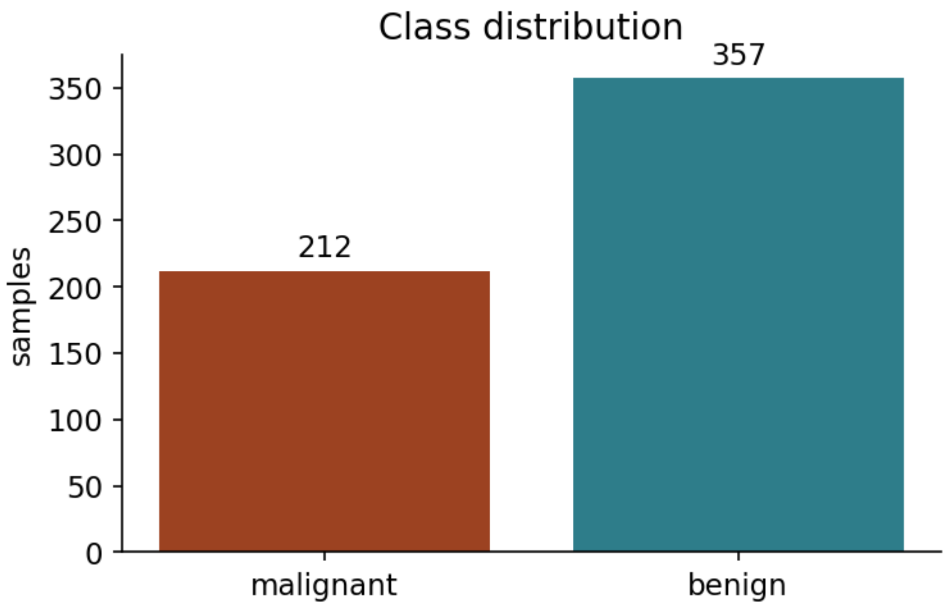
This is how modern large models are trained, and it explains their scale: text and images exist in enormous quantities, and this technique needs no annotation. We do not pursue it further here, but you should recognise it.

---

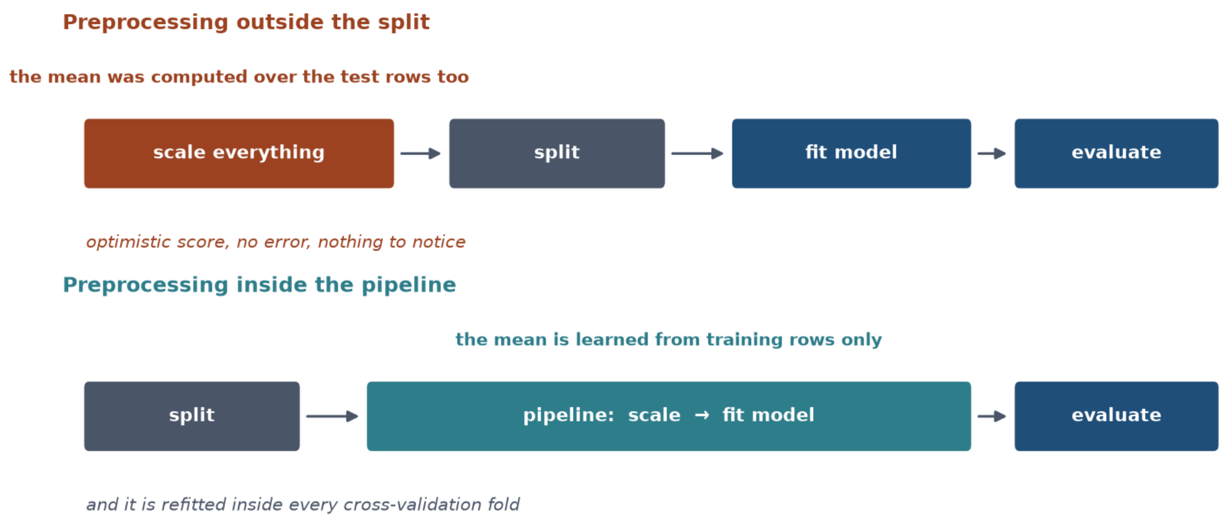
6. The end-to-end workflow



The cycle, and the order that matters most: the split comes before anything is fitted, not after.



Look before you touch anything. The class balance decides which metrics will mean something later.



The same steps, done by hand and inside a pipeline. Only one of them survives cross-validation honestly.

The order of these steps is not a convention. Several of them are only valid in this order.

1. Frame the problem. What is predicted, from what, and what would make an error harmful? Which mistake is worse? Answer before modelling — the answers determine which metric is honest, and a metric chosen after seeing results is a metric chosen to flatter them.

2. Obtain and inspect the data. Size, types, missing values, class balance, obvious errors. Look before transforming.

3. Split. Hold out a test set **now**, before anything is learnt. Every subsequent decision uses the training portion only. See Sections 2.2-2.3 for why this is the load- bearing step.

4. Baseline. The simplest thing that could work: predict the majority class, predict the mean, threshold a single variable. Without this, a score has no meaning — you cannot tell 95% accuracy that is excellent from 95% that is worse than answering "no" every time.

5. Preprocess and model, inside a pipeline. Scaling, encoding and imputation all *learn* from data (a mean, a set of categories), so they must be fitted on training data only. A pipeline enforces this structurally. Doing it by hand works until the day it does not, and that day is silent.

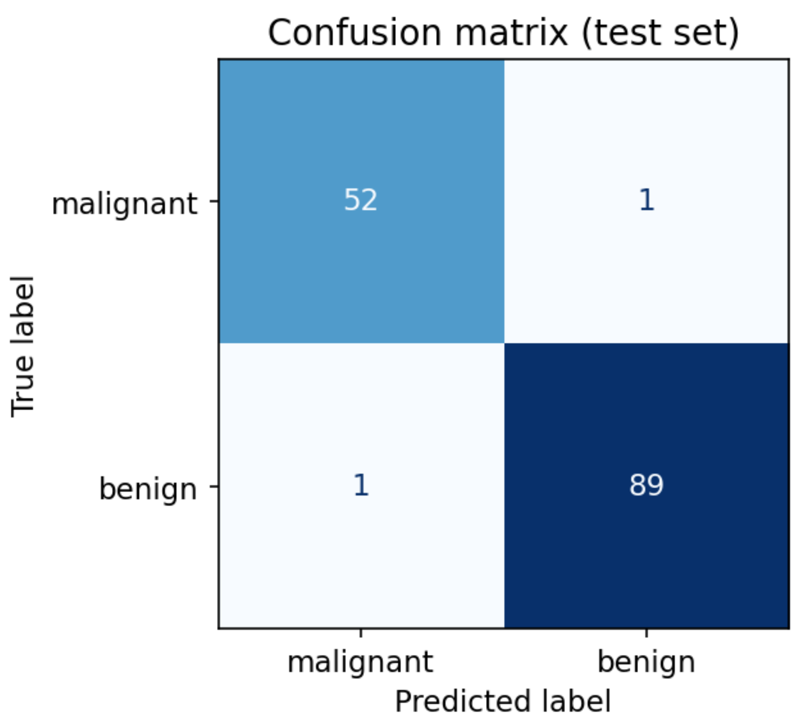
6. Evaluate with a metric that matches the cost of being wrong. Accuracy weights every error equally. Real problems rarely do.

7. Diagnose. Where does it fail, and is the failure systematic? Error analysis tells you more than a decimal place of accuracy.

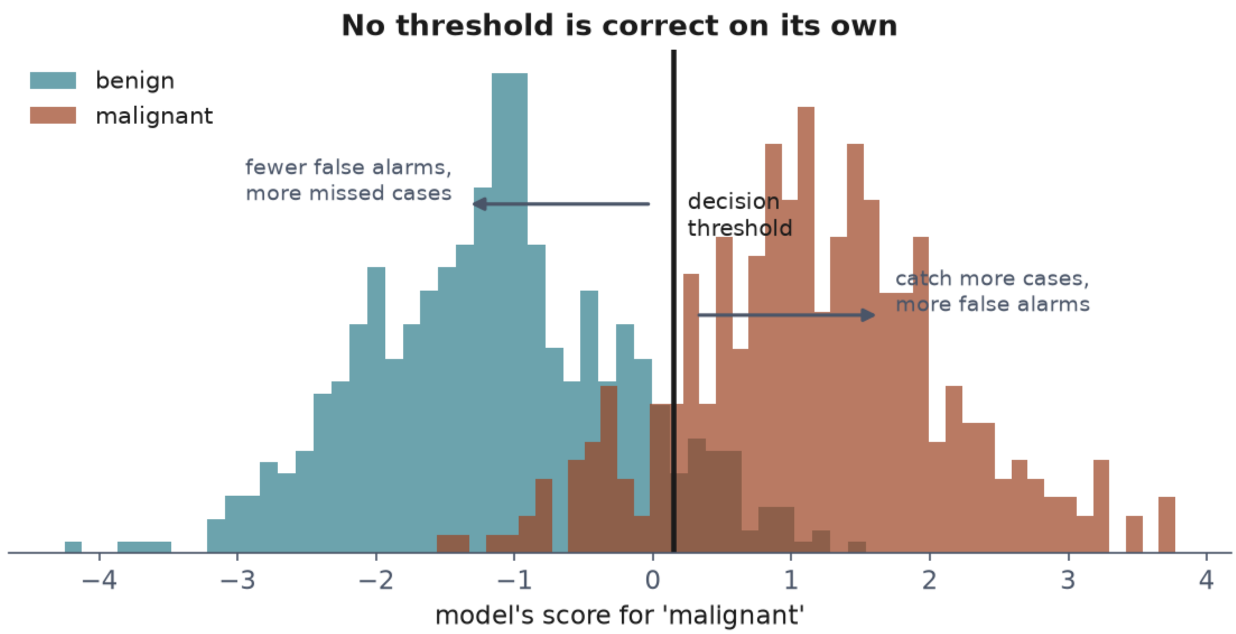
8. Iterate — with discipline. Each look at the test set to guide a decision spends some of its independence. Keep a validation set for choices, and reserve the test set for the end. Lesson 5 makes this precise.

---

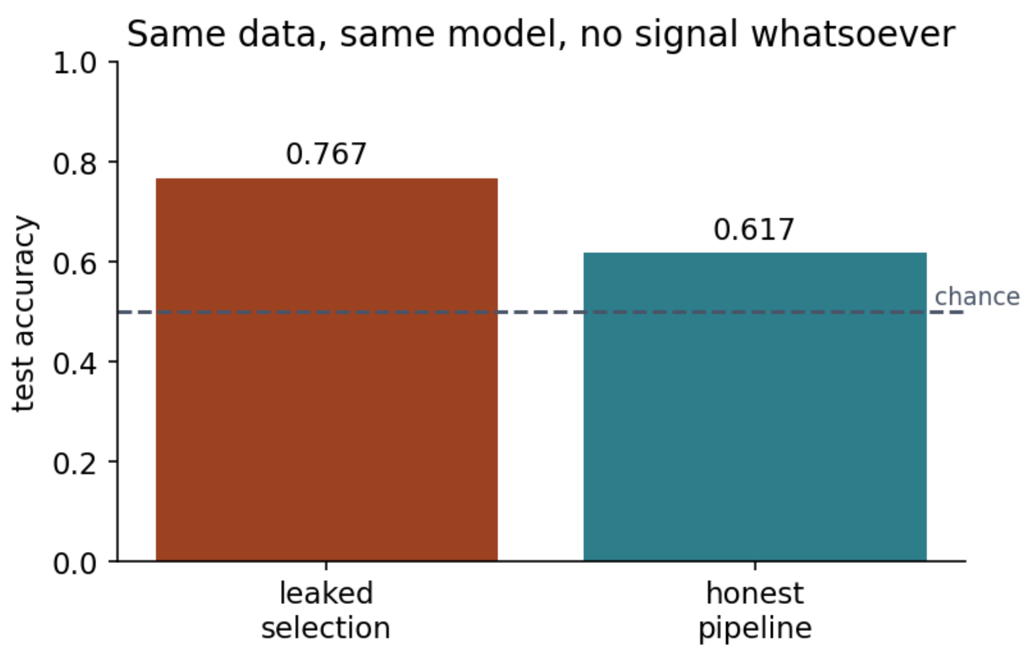
7. How models mislead



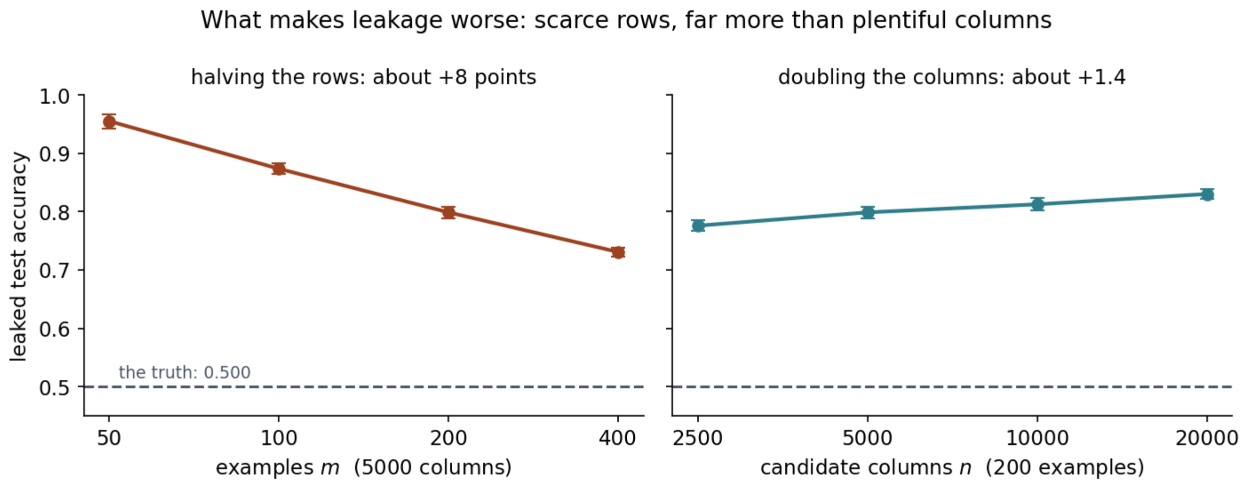
What a single accuracy figure hides: four outcomes collapsed into one number. Lesson 4 takes this apart properly.



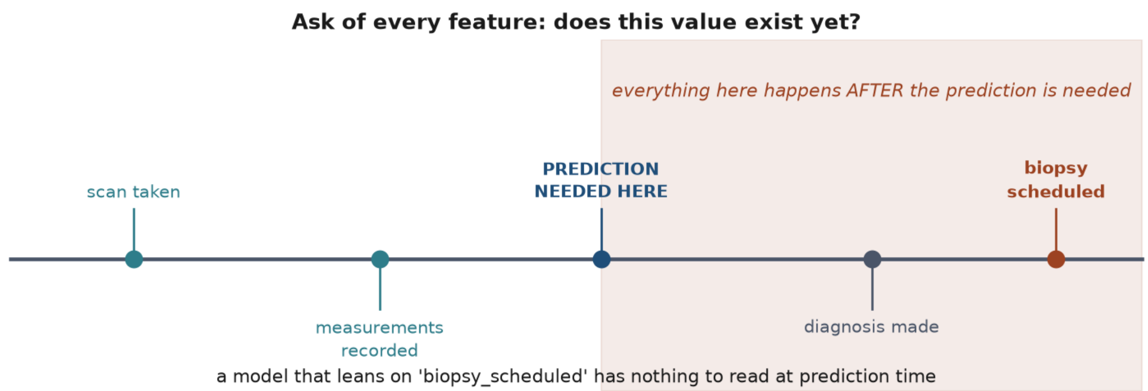
The trade every classifier makes, and the reason one number cannot describe it.



Selecting features before splitting: the test rows have already influenced which features exist, so the score that follows is not a measurement of anything.



How much worse it gets, measured over 100 repetitions per point. Read the two panels against each other rather than separately: halving the rows costs about eight points, doubling the columns about one and a half. Leakage is driven by scarce data far more than by plentiful features — and scarce data is where it is hardest to detect.



A shortcut feature — something that predicts the label in this dataset for a reason that will not survive contact with new data.

Four failure modes, all producing numbers that survive casual review, all demonstrated in notebook 03.

Data leakage. Information from the test set reaches the training procedure. Selecting features on the full dataset, scaling before splitting, or including a column recorded after the outcome. Leakage inflates scores without any error being visible in the code: notebook 03 obtains 77% accuracy on data that is pure random noise.

The figure above measures how that 77% moves. Both handles push the same way, but not equally: with 5000 candidate columns, going from 200 examples down to 100 costs about eight points of honesty, while going from 5000 columns up to 10000 costs about one and a half. The asymmetry follows from how each side scales — a noise correlation has spread $1/\sqrt{m}$, so halving the examples multiplies it by 1.41, whereas the largest of n such draws grows like $\sqrt{2\ln n}$, so doubling the columns multiplies it by 1.04. More columns buy more lottery tickets; fewer rows make each ticket pay better. The uncomfortable consequence is that leakage is at its most severe exactly where data is scarce, which is also where it is least likely to be caught.

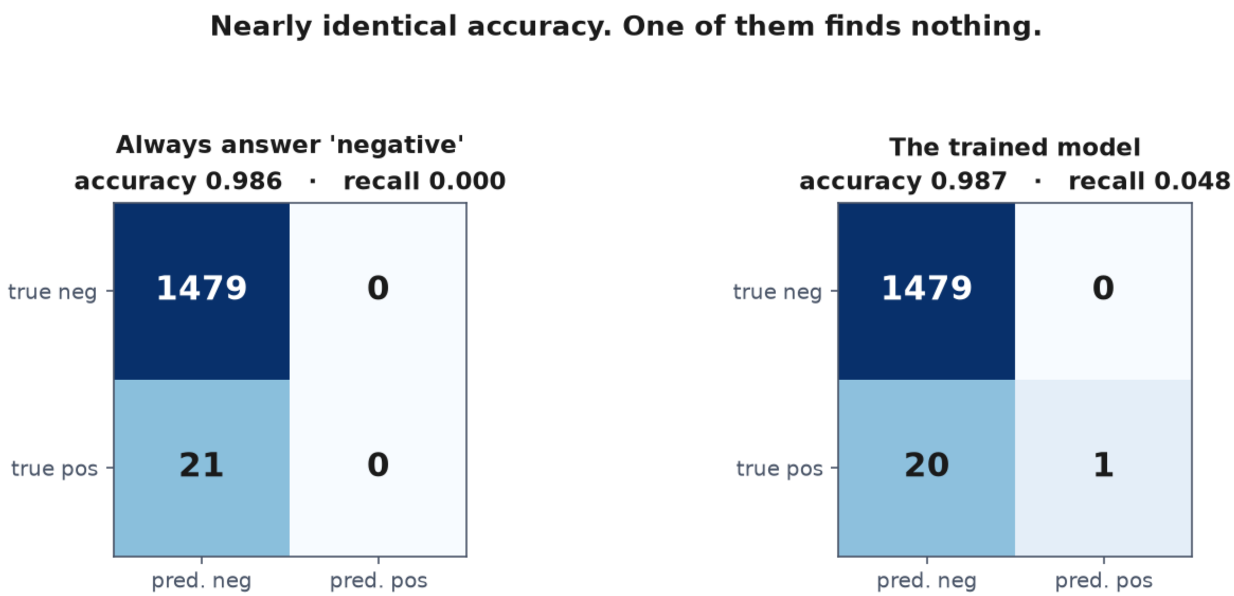
Imbalanced classes. When 1% of cases are positive, answering "negative" always scores 99%. Accuracy measures the class you are not interested in.

Shortcut features. A column that is a consequence of the label rather than a predictor of it — a treatment code, a follow-up appointment. The model leans on it and collapses in deployment, where the column does not exist yet. No metric detects this; only knowing how the data was recorded does.

Single-split noise. One train/test split gives one draw from a distribution. On a small dataset the spread across splits can exceed the difference between two competing models. A result quoted with no indication of variability is incomplete.

---

8. Limits and responsibility



99% accurate and detecting nothing. On a rare-event problem, accuracy measures the majority class and little else.

Three things worth stating on day one, since they shape everything that follows.

A model is a compressed summary of its training data, including its injustices. If past decisions discriminated, a model fitted to them will too — and it will do so with the appearance of objectivity, which makes the discrimination harder to challenge than if a person had made it.

Correlation is what these methods find. Nothing in empirical risk minimisation distinguishes a cause from a coincidence that happens to predict well in this sample. A model can be highly accurate and completely wrong about *why*, which is why it can fail abruptly when conditions change.

Accuracy is not the only property that matters. Whether a decision can be explained to the person affected, whether errors are recoverable, and who bears the cost of being wrong are all engineering requirements, not afterthoughts. They belong in the framing step, not in a paragraph at the end of a report.

---

9. What to do before the next lesson

1. Verify your environment: JupyterLab running, all three notebooks executing top to bottom.
2. Work through the notebooks in order — 01 for the shape of the process, 02 for the taxonomy, 03 for the failure modes.
3. Take the quiz in `Quizzes/`.
4. **Complete the homework** in `Exercises/01_first_workflow.md`, due at the start of Lesson 2.

Notation used in this lesson

Symbol	Meaning
x, X	one input, the design matrix
y, $\hat{y}$	true target, prediction
f	the function the model computes
S, T	the sample used for training, the test set
m, n	number of examples, number of features
L	the loss on a single example
R, $\hat{R}$	the expected risk, the empirical risk
$\mathcal{D}$	the unknown distribution the data is drawn from
p	a measured accuracy, read as a proportion (Section 2.4)
SE	the standard error of that proportion

These carry the same meanings in every lesson of the course. Where a later lesson needs a symbol for something else, it says so in its own table.

Further reading

Resource	Type	Why read it
scikit-learn: common pitfalls	Official docs	The library's own account of leakage and how pipelines prevent it
Turing, *Computing Machinery and Intelligence* (1950)	Paper	The imitation game in Turing's words; shorter and sharper than its reputation
Wolpert & Macready, *No Free Lunch Theorems* (1997)	Paper	Why no single method is best on every problem — the formal statement behind Lesson 6
Sculley et al., *Hidden Technical Debt in ML Systems* (2015)	Paper	What goes wrong once a model leaves the notebook

Resource	Type	Why read it
Géron, *Hands-On Machine Learning*, ch. 1–2	Book	A complementary treatment of the workflow, with a different worked example
`Resources/history_of_ai.md`	Course material	The historical arc in more depth, with primary sources